

Embedded Systems Laboratory Report

FreeRTOS Task Synchronisation on the B-L475E-IOT01A

Basic Problem and Option Problem A

YU-CHEN, CHU
Student ID: R12942157

September 21, 2026

Source code: https://github.com/oldpig123/embedded_system/tree/master/lab1

Contents

1	Introduction	2
2	Development Environment	2
3	Basic Problem	2
3.1	Requirements	2
3.2	System Architecture	2
3.3	Synchronisation Objects	3
3.4	Timing Design	3
3.5	Implementation	3
3.6	Results	4
4	Option Problem A: Long Press Detection	4
4.1	Requirements	4
4.2	Detecting Press Duration	4
4.3	Why a Queue Rather Than a Semaphore	5
4.4	Implementation	5
4.5	Results	6
5	Problems Encountered	6
5.1	A misspelled callback silently does nothing	6
5.2	Busy-waiting in a high-priority task starves everything below it	7
5.3	HAL_Delay() inside an interrupt handler deadlocks	7
5.4	osDelay() inside an interrupt handler is a silent no-op	7
5.5	A FreeRTOS task function must never return	7
6	Design Discussion	7
6.1	Signalling and mutual exclusion are different problems	7
6.2	Mutex granularity determines takeover latency	8
6.3	Sharing SysTick between the HAL and the RTOS	8
7	Conclusion	8

1 Introduction

The purpose of this exercise was to implement two independent blinking behaviours on a single LED, each driven by its own FreeRTOS task, and to coordinate them so that the two behaviours never interfere with one another. Because both tasks manipulate the same physical output pin, the exercise is fundamentally about *synchronisation*: deciding which task is allowed to drive the LED at any given moment, and how an asynchronous external event (a button press, or a periodic timer expiry) is communicated to the task that should react to it.

Two related but distinct mechanisms are used throughout. A *binary semaphore* and a *message queue* are used for *signalling*: telling a sleeping task that an event has occurred. A *mutex* is used for *mutual exclusion*: guaranteeing that only one task drives the shared LED at a time. Distinguishing these two roles turned out to be the central conceptual lesson of the exercise.

The extension (Option Problem A) adds the requirement that the button distinguish a normal press from a long press, which forces the signalling mechanism to carry *data* rather than a bare notification, and therefore replaces the semaphore with a message queue.

2 Development Environment

Item	Detail
Board	B-L475E-IOT01A Discovery kit
MCU	STM32L475VG (ARM Cortex-M4F)
RTOS	FreeRTOS, accessed through the CMSIS-RTOS v2 API
Code generation	STM32CubeMX
Toolchain	arm-none-eabi-gcc, CMake build
Indicator	LED2 on PB14
Input	User button on PC13, via EXTI line 13

Table 1: Development environment.

All RTOS objects are created with the CMSIS-RTOS v2 API (`osThreadNew`, `osSemaphoreNew`, `osMutexNew`, `osTimerNew`, `osMessageQueueNew`) rather than calling the underlying FreeRTOS functions directly.

3 Basic Problem

3.1 Requirements

Two tasks share LED2 and must not interfere with each other:

- **Task_1** — pressing the user button triggers LED2 to blink for 5 s at 1 Hz. The EXTI interrupt handler detects the press and informs Task_1 through a binary semaphore.
- **Task_2** — LED2 blinks for 2 s at 10 Hz, repeating every 10 s. A timer interrupt detects the time-up event and informs Task_2 through a second binary semaphore.

A mutex or semaphore must protect the blinking procedures so that while one blinking sequence is in progress, the other waits.

3.2 System Architecture

Both tasks follow the same three-stage pattern:

external event → ISR / callback → signalling object → task performs blinking

The interrupt or callback performs no work beyond signalling; all timing and GPIO manipulation happens in task context. This separation is essential, because delaying inside an interrupt handler is either unsafe or ineffective (Section 5).

Role	Function	Priority	Trigger	Blink pattern
Task_1	StartTask03	osPriorityHigh7	EXTI13 ISR	5 s @ 1 Hz
Task_2	StartDefaultTask	osPriorityNormal	periodic RTOS timer	2 s @ 10 Hz

Table 2: Task structure. A third thread, StartTask02, remains from an earlier exercise and is idle.

3.3 Synchronisation Objects

Object	Creation	Purpose
myMutex01Handle	osMutexNew(NULL)	protects LED2
myBinarySem02Handle	osSemaphoreNew(1, 0, NULL)	timer → Task_2
myTimer01Handle	osTimerNew(..., osTimerPeriodic, ...)	10 s periodic tick
myQueue01Handle	osMessageQueueNew(16, sizeof(uint32_t), NULL)	ISR → Task_1

Table 3: RTOS objects. The queue replaces Task_1's binary semaphore in Option Problem A.

All objects are created in `main()` after `osKernelInitialize()` and before `osKernelStart()`. Both semaphores are created with an initial count of zero, so that the waiting task genuinely blocks until a real event occurs rather than passing through immediately on its first attempt.

3.4 Timing Design

A blink of frequency f has period $T = 1/f$, and a 50 % duty cycle is obtained by toggling the pin every $T/2$. The number of loop iterations required for a total duration D is therefore $N = D/T$.

Sequence	Frequency	Half period	Duration	Iterations
Task_2 burst	10 Hz	50 ms	2 s	20
Task_1 normal press	1 Hz	500 ms	5 s	5
Task_1 long press	10 Hz	50 ms	5 s	50

Table 4: Blink timing parameters.

Each loop iteration contains two toggles and two delays, so one iteration corresponds to one full on/off cycle.

3.5 Implementation

The timer callback does nothing but release the semaphore that Task_2 is waiting on:

```

1 void Timer01_Callback(void *argument)
2 {
3     osSemaphoreRelease(myBinarySem02Handle);
4 }

```

Listing 1: Periodic timer callback signalling Task_2.

Task_2 blocks on that semaphore and performs its burst when signalled. Note that the mutex is acquired and released around each individual toggle rather than around the whole burst; the reason is discussed in Section 6.2.

```

1 void StartDefaultTask(void *argument)
2 {
3     for(;;)
4     {
5         osSemaphoreAcquire(myBinarySem02Handle, osWaitForever);
6         for(int i = 0; i < 20; i++)
7         {
8             osMutexAcquire(myMutex01Handle, osWaitForever);
9             HAL_GPIO_TogglePin(GPIOB, LED2_Pin);
10            osMutexRelease(myMutex01Handle);
11            osDelay(50);
12            osMutexAcquire(myMutex01Handle, osWaitForever);
13            HAL_GPIO_TogglePin(GPIOB, LED2_Pin);
14            osMutexRelease(myMutex01Handle);
15            osDelay(50);
16        }
17    }
18 }

```

Listing 2: Task_2: 2 s at 10 Hz, triggered by the timer.

Because the task no longer performs its own inter-burst delay, the 10-second period is governed entirely by the timer. The task simply returns to blocking on the semaphore as soon as a burst finishes.

3.6 Results

The observed behaviour is a 2-second burst of fast blinking followed by 8 seconds of darkness, repeating indefinitely — a 10-second period, as required.

One initially surprising detail is that nothing happens for the first 10 seconds after reset. This is correct: the periodic timer starts counting when `osTimerStart()` executes, so its first expiry occurs a full period later. Until then Task_2 is blocked on a semaphore whose count is zero. Should an immediate first burst be desired, releasing the semaphore once manually before `osKernelStart()` would shift the phase without altering the period.

4 Option Problem A: Long Press Detection

4.1 Requirements

The button must distinguish two events: a normal press, and a long press in which the button is held for more than one second. A normal press produces the original 5 s at 1 Hz blink; a long press produces a 5 s at 10 Hz blink. Because Task_1 must now be told *which* of two events occurred, the specification requires a message queue rather than a binary semaphore.

4.2 Detecting Press Duration

Measuring how long a button is held requires knowing both when it was pressed and when it was released. The original configuration generated an interrupt only on the falling edge, so the release was invisible to software. The EXTI configuration for PC13 was therefore changed from `GPIO_MODE_IT_FALLING` to `GPIO_MODE_IT_RISING_FALLING` in STM32CubeMX and the project regenerated.

With both edges enabled the same callback fires twice per press, so the two cases must be told apart by reading the current pin level. The button is active low, so a level of `GPIO_PIN_RESET`

means the press has just occurred and `GPIO_PIN_SET` means the button has just been released. On press, the current tick is recorded; on release, the elapsed time is compared against the one-second threshold and the corresponding message is posted to the queue.

The timestamp must survive between two separate invocations of the interrupt handler, so it is a file-scope `volatile` variable rather than a local.

4.3 Why a Queue Rather Than a Semaphore

A binary semaphore carries no payload. It can only communicate the fact that *something* happened, which is sufficient when there is exactly one kind of event. Once the button can produce two distinguishable events, the signalling mechanism must carry a value. A message queue provides exactly this: the interrupt posts a value, and the receiving task both blocks until a message arrives and learns which event it was. The queue therefore subsumes the semaphore's role, and `Task_1`'s binary semaphore was removed entirely.

4.4 Implementation

```

1 void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
2 {
3     if (GPIO_Pin == GPIO_PIN_13)
4     {
5         if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13) == GPIO_PIN_RESET)
6         {
7             /* falling edge: button pressed, record the time */
8             button_press_time = HAL_GetTick();
9         }
10        else
11        {
12            /* rising edge: button released, classify the press */
13            if (HAL_GetTick() - button_press_time >= 1000)
14            {
15                uint32_t message = 1; /* long press */
16                osMessageQueuePut(myQueue01Handle, &message, 0, 0);
17            }
18            else
19            {
20                uint32_t message = 0; /* short press */
21                osMessageQueuePut(myQueue01Handle, &message, 0, 0);
22            }
23        }
24    }
25 }

```

Listing 3: EXTI callback: classifying press duration and posting a message.

The final argument of `osMessageQueuePut()` is the timeout, and it must be zero when the function is called from interrupt context. The CMSIS-RTOS v2 wrapper detects that it is running inside an interrupt and rejects any non-zero timeout outright, because an interrupt handler cannot block.

`Task_1` blocks on the queue and selects its blinking pattern from the received value:

```

1 void StartTask03(void *argument)
2 {
3     for(;;)
4     {
5         uint32_t message;
6         osMessageQueueGet(myQueue01Handle, &message, NULL, osWaitForever);
7
8         if (message == 1) /* long press: 5 s at 10 Hz */
9         {

```

```

10     osMutexAcquire(myMutex01Handle, osWaitForever);
11     for (int i = 0; i < 50; i++)
12     {
13         HAL_GPIO_TogglePin(GPIOB, LED2_Pin);
14         osDelay(50);
15         HAL_GPIO_TogglePin(GPIOB, LED2_Pin);
16         osDelay(50);
17     }
18     osMutexRelease(myMutex01Handle);
19 }
20 else                                     /* normal press: 5 s at 1 Hz */
21 {
22     osMutexAcquire(myMutex01Handle, osWaitForever);
23     for (int i = 0; i < 5; i++)
24     {
25         HAL_GPIO_TogglePin(GPIOB, LED2_Pin);
26         osDelay(500);
27         HAL_GPIO_TogglePin(GPIOB, LED2_Pin);
28         osDelay(500);
29     }
30     osMutexRelease(myMutex01Handle);
31 }
32 }
33 }

```

Listing 4: Task_1: selecting a blink pattern from the queue message.

4.5 Results

A short press produces five slow blinks over five seconds; holding the button for more than one second and then releasing produces fifty fast blinks over the same five seconds. In both cases the periodic background burst from Task_2 is correctly excluded for the duration of the sequence.

One known limitation is that no debouncing is performed. Mechanical contact bounce can in principle generate spurious edge interrupts at the moment of press or release, which would corrupt a duration measurement. In practice the one-second threshold is long enough that misclassification was not observed, but a production implementation would filter the input, either in software or with an RC network.

5 Problems Encountered

The majority of the effort in this exercise went into diagnosing behaviour that appeared to be "nothing happening". Each case had a distinct cause, and together they illustrate most of the important constraints of programming under an RTOS.

5.1 A misspelled callback silently does nothing

The first version of the interrupt handler was named `HAL_GPIO_EXIT_Callback` instead of `HAL_GPIO_EXTI_Callback`. The code compiled and linked without any warning, but the button had no effect.

The HAL declares its callbacks as *weak* symbols with empty bodies, so that the application can override them simply by defining a function of the same name. A misspelled name does not override anything; it merely defines an unused function, and the empty default implementation continues to be called. The lesson is that a silent failure of this kind is characteristic of the weak symbol mechanism, and that callback names must be copied exactly.

5.2 Busy-waiting in a high-priority task starves everything below it

An early version used `HAL_Delay()` inside both tasks. The higher-priority task blinked correctly while the lower-priority task never ran at all.

`HAL_Delay()` does not block in the RTOS sense: it spins in a loop polling `HAL_GetTick()` and therefore remains permanently *runnable*. Under a preemptive priority scheduler, a runnable high-priority task is always selected over a lower-priority one, so the lower-priority task is never scheduled. Replacing `HAL_Delay()` with `osDelay()` fixed this: `osDelay()` moves the calling task into the blocked state, allowing the scheduler to run other tasks during the delay.

5.3 `HAL_Delay()` inside an interrupt handler deadlocks

A later attempt performed the blinking loop directly inside the EXTI callback using `HAL_Delay()`. Pressing the button caused the LED to switch on and freeze permanently.

The cause is an interaction between interrupt priorities. The EXTI interrupt is configured at NVIC priority 5, whereas the SysTick interrupt that increments the HAL tick runs at priority 15 — a lower priority on Cortex-M, where a numerically larger value is less urgent. While the EXTI handler is executing, SysTick cannot preempt it, so `HAL_GetTick()` never advances. `HAL_Delay()` waits for a tick value that can only be produced by an interrupt it is itself blocking: a deadlock.

5.4 `osDelay()` inside an interrupt handler is a silent no-op

Replacing `HAL_Delay()` with `osDelay()` in the same handler removed the freeze but produced no visible blinking at all.

Inspection of the CMSIS-RTOS v2 wrapper shows that `osDelay()` begins by testing whether it is executing in interrupt context, and if so returns `osErrorISR` immediately without delaying. Every delay in the loop therefore became a no-op, the ten toggles executed within a few microseconds, and since ten is an even number the LED finished in the state it started in. Blocking operations simply cannot be performed in an ISR, regardless of which API is used.

Taken together, the two previous problems establish the rule that an interrupt handler must only *signal* a task and return, never perform timed work itself.

5.5 A FreeRTOS task function must never return

While restructuring the code, one task body was temporarily commented out, including its enclosing `for(;;)` loop. The whole system stopped responding.

A task function is not an ordinary function that the scheduler calls repeatedly; it is an entry point that must never return. When it does, execution continues into whatever instructions follow it in memory, which in this case caused a fault and left the CPU spinning in the default `HardFault_Handler`, halting every task. Because the affected task had the highest priority, it was scheduled first and the system failed almost immediately after `osKernelStart()`.

A related misconception is worth recording: calling a task's function by name from an interrupt handler does not start that task. It executes the function inline, in interrupt context, on the interrupt's stack — and if the function contains an infinite loop, the handler never returns at all.

6 Design Discussion

6.1 Signalling and mutual exclusion are different problems

It is tempting to treat semaphores and mutexes as interchangeable, since both can be used to make a task wait. They solve different problems. The semaphore (or queue) answers “*has an*

event occurred?” and is signalled by one party and consumed by another. The mutex answers “*may I touch the shared resource?*” and is always released by the same task that acquired it. This design needs both, for different reasons, and the two cannot be merged without losing one of the guarantees.

An alternative to the mutex would have been a plain `volatile` flag checked before each toggle. This works for a single boolean on Cortex-M, where aligned word accesses are atomic, but it requires the other task to poll the flag rather than block, and it offers no protection for anything more complex than a single variable. The mutex expresses the intent directly and lets the RTOS handle the blocking.

6.2 Mutex granularity determines takeover latency

A mutex provides mutual exclusion, not preemption. It cannot interrupt a task that already holds the lock; a waiting task can only be admitted once the holder releases. The consequence is that the *size of the critical section* directly determines how quickly one task can take over from another.

In the first implementation, Task_2 acquired the mutex once and held it for its entire 2-second burst. A button press arriving early in that burst had to wait up to two full seconds before Task_1’s sequence could begin, which was clearly visible as a delayed and apparently unresponsive button.

Narrowing the critical section so that the mutex is held only around each individual `HAL_GPIO_TogglePin()` call, and released before the `osDelay()`, reduced the worst-case wait to approximately one toggle period (50 ms) — indistinguishable from instantaneous to a human observer. Task_1 still holds the mutex for its whole sequence, so once it takes over it cannot be interrupted in turn.

The resulting behaviour is best described as *pause and resume*: Task_2’s loop counter is preserved while it waits for the mutex, so after Task_1 finishes, Task_2 completes whatever remained of its interrupted burst rather than restarting. Discarding the remainder instead would require an explicit cancellation mechanism, such as a flag checked inside Task_2’s loop, which was judged unnecessary for this specification.

6.3 Sharing SysTick between the HAL and the RTOS

STM32CubeMX warns that when an RTOS is used, the HAL timebase should be moved off SysTick onto a separate hardware timer. The reason is that FreeRTOS takes over the SysTick interrupt for its own scheduling tick, while the HAL relies on the same interrupt to maintain `HAL_GetTick()`. Inside a FreeRTOS critical section the tick interrupt may be delayed, so HAL timeouts and delays can measure time inaccurately. This project retains the default shared configuration, which is acceptable here because no HAL timing function is used after the scheduler starts, but it would be worth changing in any design that relies on HAL peripheral timeouts at runtime.

7 Conclusion

Two tasks were implemented that drive the same LED with different blinking patterns, one triggered by an external interrupt and one by a periodic RTOS timer, with a mutex guaranteeing that the two sequences never overlap. The extension added press-duration measurement using both edges of the EXTI line and replaced the binary semaphore with a message queue, allowing a single task to select between two blinking patterns based on the value it receives.

The most valuable outcome was not the final code but the set of failure modes encountered along the way. Each of them — the unoverridden weak callback, the starvation caused by busy-waiting, the deadlock caused by delaying inside a high-priority interrupt, the silently ignored `osDelay()` in an ISR, and the task that returned — reflects a specific property of how an RTOS

schedules work. Collectively they reinforce a single design principle: interrupt handlers should capture events and defer all real work to tasks, which are the only context in which blocking, timing and resource arbitration can be performed correctly.